University of Pittsburgh

CS 2410 Introduction

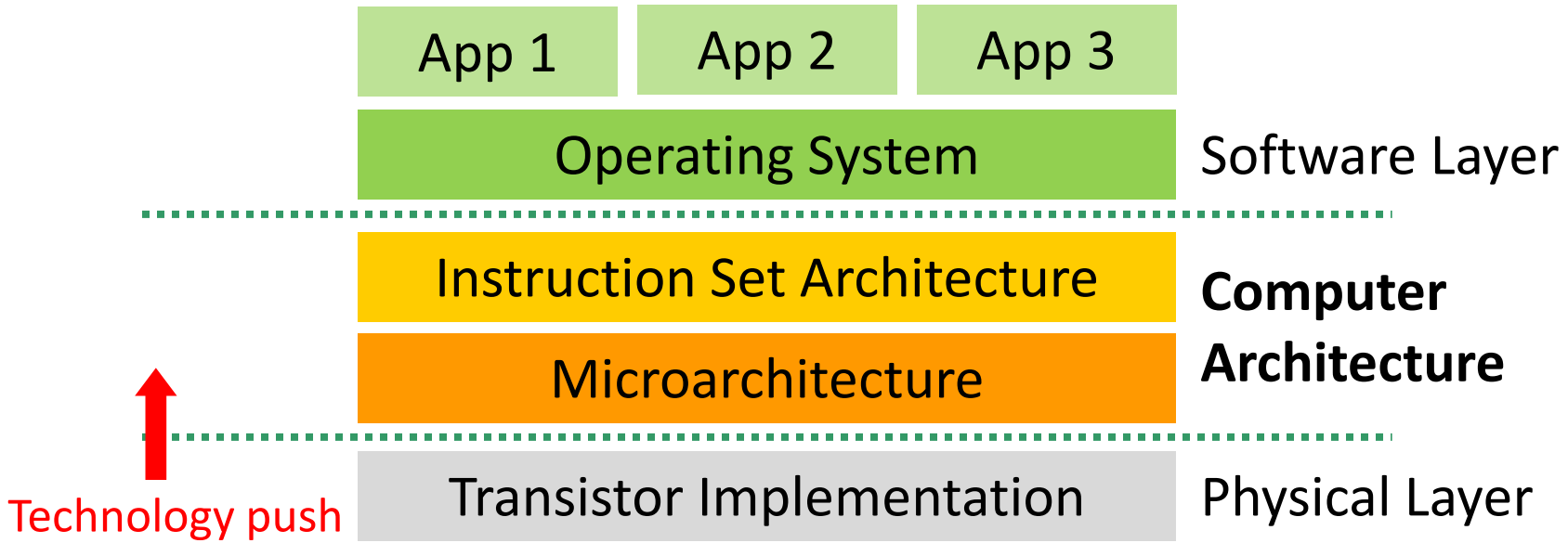
Technology Constraints

Wonsun Ahn
Department of Computer Science
School of Computing and Information

Technology Constraints

Technology Constraints

- Constraints in technology push architecture too



  - *Power Wall*: Thermal Design Power (TDP) constraint
  - *Memory Wall*: Constraint in bandwidth to memory
  - *Variability*: Limits in the precision of manufacturing technology
- Processor must be designed to meet all constraints

Technology Constraints

Power Wall

Power Wall

- $Power_{CPU} = Power_{dynamic} + Power_{leakage}$
  $Power_{dynamic} \propto A * N * CFV^2$
  $Power_{leakage} \propto f(N, V, V_{th}) \propto N * V * e^{-Vth}$
  - Leakage power is also called *static power*
  - This total CPU power cannot exceed TDP

- Moore's Law transistor scaling means two things:
  - N = Number of transistors ($\propto$ 1/transistor size^2) ⇧ ⇧
  - C = Capacitance ($\propto$ transistor size) ⇩
  - Reductions in C does not compensate for increases in N

- Architects must use tricks to keep power in check
  - To keep packing more transistors to increase performance

1. Reducing Dynamic Power

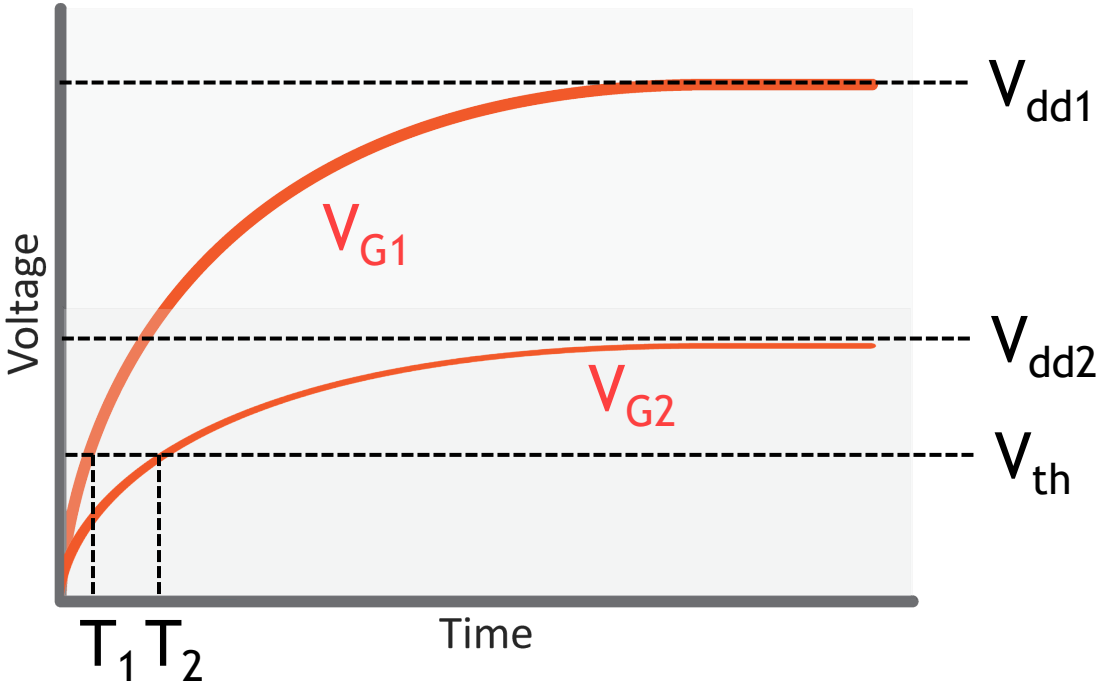
- $Power_{dynamic}$ ($\propto A * N * CFV^2$) + $Power_{leakage}$ ($\propto N * V * e^{-Vth}$)

- Reducing A (Activity): *Clock gating*
  - Disables the clock signal to unused parts of the chip (idle cores)
  - Wake-up is instantaneous (the moment clock signal goes in)

- Reducing F (Frequency) and V (Supply Voltage)
  - When F is reduced, V can also be reduced
    (The RC curve and water pressure, remember?)
  - *Dynamic Voltage Frequency Scaling (DVFS)* done on multi-cores
    - Slow down low-priority cores, speed up high-priority cores

DVFS and Transistor Speed

- RC Charging Curve of V_G



- Delay (T) increases with supply voltage (V_{dd}) decrease → Frequency (1/T) decreases with V_{dd} decrease

2. Reducing Leakage Power

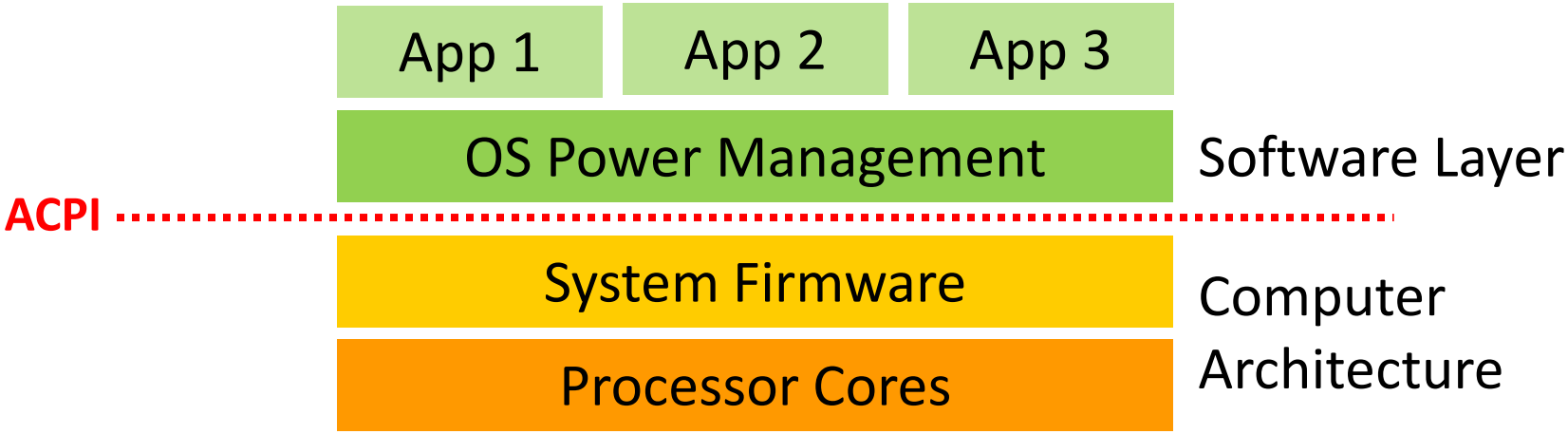
- $Power_{dynamic}$ ($\propto A * N * CFV^2$) + $Power_{leakage}$ ($\propto N * V * e^{-Vth}$)

- Reducing N (Transistor Number): *Power gating*
  - Disables power to unused parts of the chip (unused cores)
  - Eliminates dynamic power *and* leakage power to those parts
  - Drawback: wake-up takes a much longer time than clock gating
    - Delay for supply voltage to stabilize
    - Delay to backup and restore CPU state to/from memory

- Reducing V (Supply Voltage): *DVFS* also helps here
  - Only the voltage scaling part, not the frequency scaling

OS Manages Power

- Who decides which cores to clock gate and power gate?
- Who decides how to apply DVFS to the cores?
- *ACPI* (*Advanced Configuration and Power Interface*)
  - OS performs power management using this interface
    - OS knows best which threads to prioritize for best user experience
  - Open standard interface to system firmware
    - Firmware sends signals to processor cores to control them



3. Simpler Processor Design

- Diminishing returns on performance with more FUs
  - Data dependence means more FUs are idle while spending energy
- Control logic to orchestrate FUs is power hungry
  - FUs need orchestration so they don't violate data dependence

⇨ Push towards simpler architectures:

- *Multi-cores*: Run multiple programs (threads) on simple cores
- *GPUs*: Run each instruction on massively parallel compute units

4. Less Data Movement

- A lot of energy is expended moving data to FUs
  - Especially if data is in off chip DRAM
    - Memory controller must work hard to match DDR data rates
    - Memory controller must drive long off chip wires to DRAM
  - Data access in registers or caches are much cheaper

⇨ Push towards less data movement:

- *Caches*: Reduces DRAM accesses. Also, minimal dynamic power.
- *PIM (Processing-In-Memory)*: Does computation in DRAM chip.

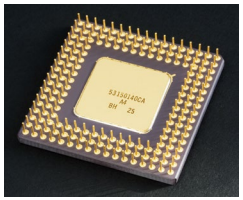
Technology Constraints

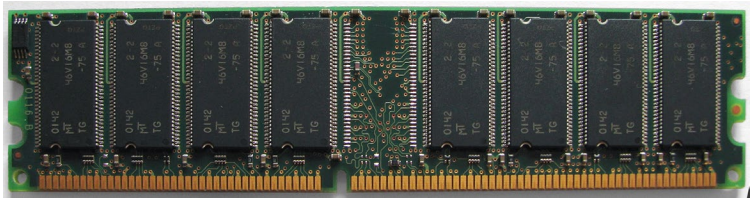
Memory Wall

Memory Wall

- Refers to both *latency (ns)* and *bandwidth (GB/s)*
  - CPU frequency and overall performance increased dramatically
  - Memory (DRAM) latency and bandwidth have lagged far behind

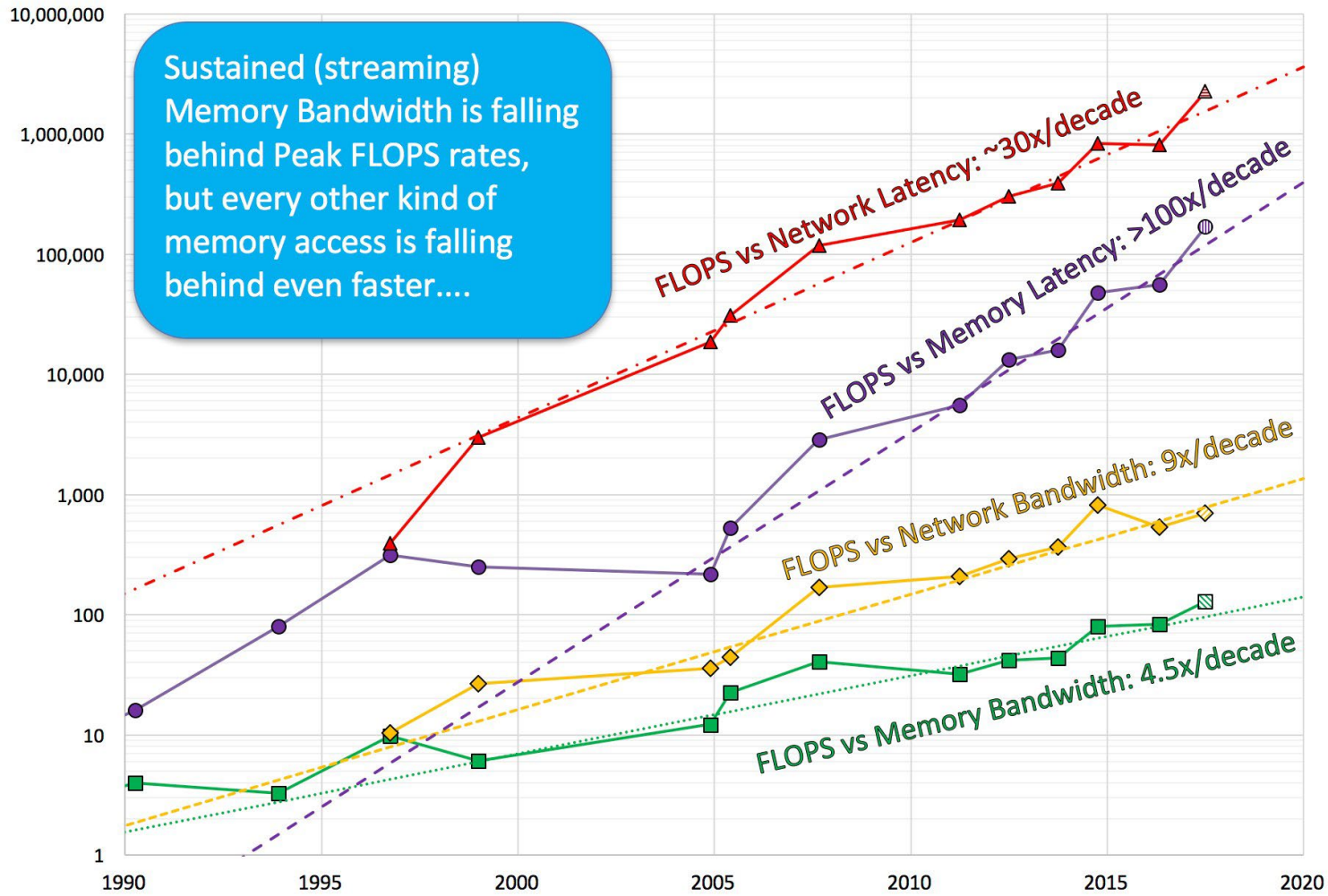
- Why?
  - Limit on the number of CPU / DRAM pins that can be soldered on





  - DRAM manufacturers have traditionally prioritized capacity
  - DDR1 (1998): 1.6 GB/s → DDR5 (2021): 38.4 GB/s
    (Impressive? Not so much compared to CPU performance)

Memory Wall

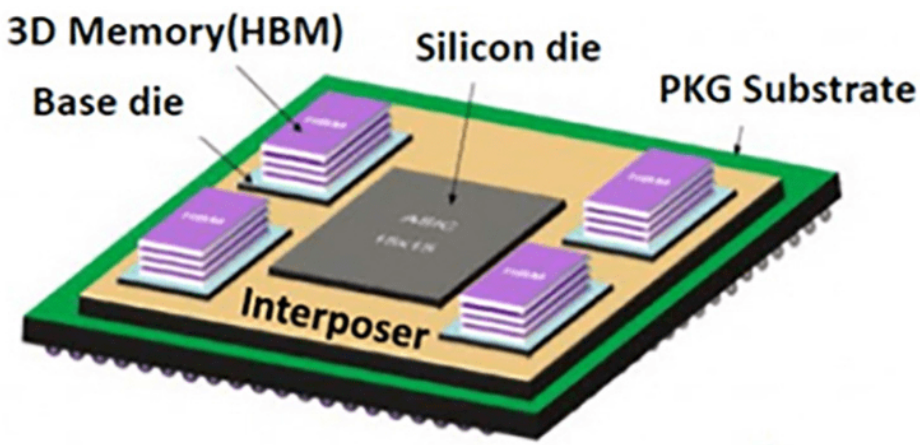


Source: SC16 Invited Talk ""Memory Bandwidth and System Balance in HPC Systems" by John D. McCalpin

- FLOPS = floating point operations per second (performance)

Memory Wall

- Where did the Memory Wall push architecture?
- *Caches*: If hit in cache, no need to go to memory
  - Caching reduces both data access latency/bandwidth
- *3D-Stacked Memory:* CPU and memory in same package
  - Interposer layer connects stacked memory with CPU die
  - Through silicon vias (TSVs) connect a stack of memory dies



Technology Constraints

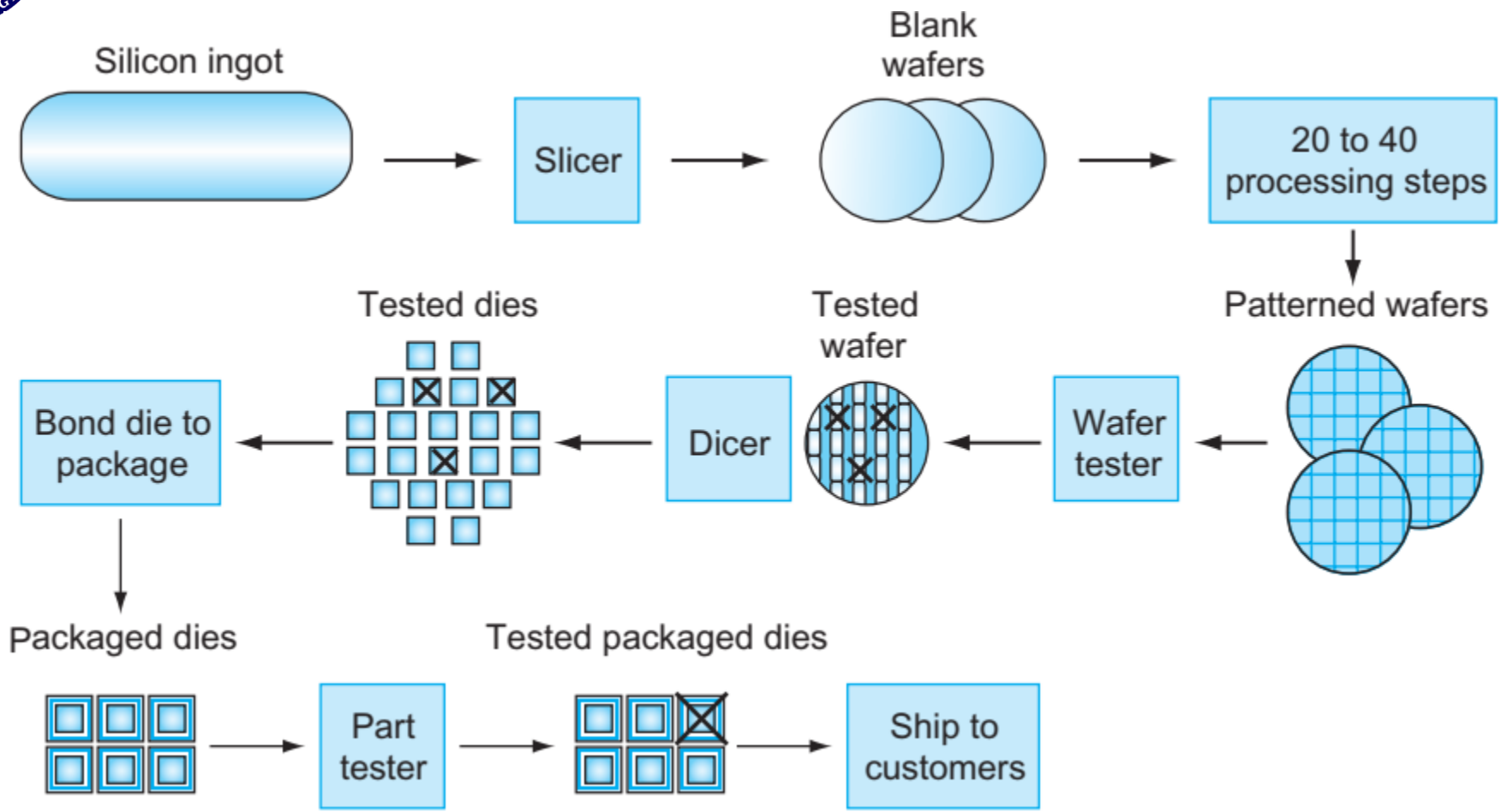
Variability

Variability

- *Variability*: differences in speed of individual transistors
  - If fab can't ensure uniformity of transistors, speeds will differ
  - Speed differences mostly come from variations in V_{th}:
    low V_{th} → cycle time ⇩ but leakage power ⇧
    high V_{th} → cycle time ⇧ but leakage power ⇩

- If unlucky and a logic path has lots of *slow* transistors
  → CPU may miss clock cycle time if path is exercised
  → CPU must be discarded, since it malfunctions
- If unlucky and a region has too many *fast* transistors
  → Region may generate too much heat due to low V_{th}
  → CPU must be discarded, due to overheating
- Leads to low chip *yield*

Wafer Yield



- Lower yield leads to higher production cost

Variability

- Where did Variability push architecture?

- *Product binning*: Sell slower CPUs at a cheaper "bin"
  - And rate slower CPUs at a lower CPU frequency
  - Instead of discarding them as "malfunctioning"
- *Multi-cores*: Easy to disable one or two buggy cores
  - Compared to single core where subcomponents must be disabled
  - When cores are disabled, sold as a cheaper chip
- *Limited pipelining*: pipelining exacerbates variability
  - With long stages, many transistors so tend to even each other out
  - With short stages, few transistors so probable all are slow

Intel i9 Product Binning

Model	# Cores	# Threads	Base Clock	All Core Turbo	Turbo Boost	Total L3 Cache	PL1 TDP
i9-10900K	10	20	3.7	4.8	5.1	20	125
i9-10900KF	10	20	3.7	4.8	5.1	20	125
i9-10900	10	20	2.8	4.5	5.0	20	65
i9-10900F	10	20	2.8	4.5	5.0	20	65
i9-10900T	10	20	1.9	3.7	4.5	20	35
i7-10700K	8	16	3.8	4.7	5.0	16	125
i7-10700KF	8	16	3.8	4.7	5.0	16	125
i7-10700	8	16	2.9	4.6	7.7	16	65
i7-10700F	8	16	2.9	4.6	4.7	16	65
i7-10700T	8	16	2.0	3.7	4.4	16	35
i5-10600K	6	12	4.1	4.5	4.8	12	125
i5-10600K	6	12	4.1	4.5	4.8	12	125
i5-10600	6	12	3.3	4.4	4.8	12	65
i5-10600T	6	12	2.4	3.7	4.0	12	35
i5-10500	6	12	3.1	4.2	4.5	12	65
i5-10500T	6	12	2.3	3.5	3.8	12	35
i5-10400	6	12	2.9	4.0	4.3	12	65
i5-10400F	6	12	2.9	4.0	4.3	12	65
i5-10400T	6	12	2.0	3.2	3.6	12	35

Why the close to 4X difference? Clock difference is just 2X!

Produced from one wafer

4 out of 10 cores were discarded.

Source: https://www.techspot.com/article/2039-chip-binning/

* TDP is calculated using the Base Clock frequency at a nominal supply voltage

Architecture Response

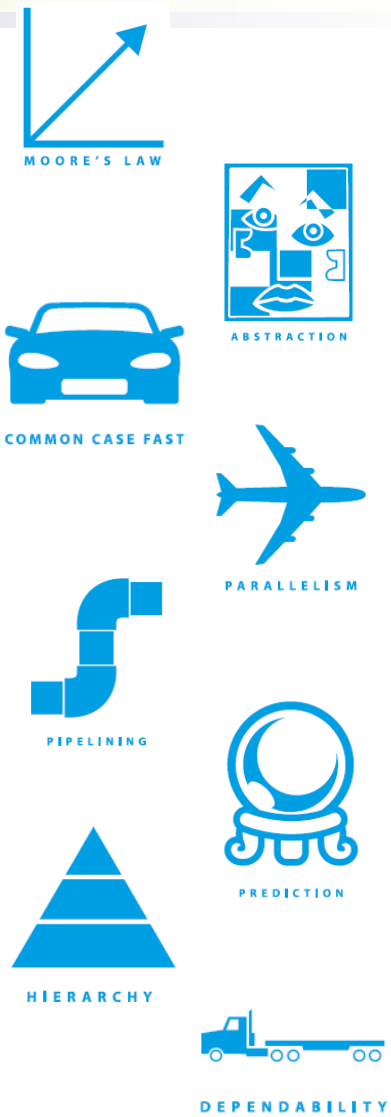
How architects have responded

Opportunities for Speed Improvement

- So Dennard Scaling is dead
  - Free CPU frequency gains are no longer there
- And we are walled in by technology constraints
  - Power wall
  - Memory wall
  - Variability
  - …
- Where do architects go look for performance?
  - Remember, Moore's Law is still giving us tons of transistors!

8 Great Ideas in Computer Architecture

- Design for *Moore's Law*
- Use *abstraction* to simplify design
- Make the *common case fast*
- Performance *via parallelism*
- Performance *via pipelining*
- Performance *via prediction*
- *Hierarchy* of memories
- *Dependability via* redundancy

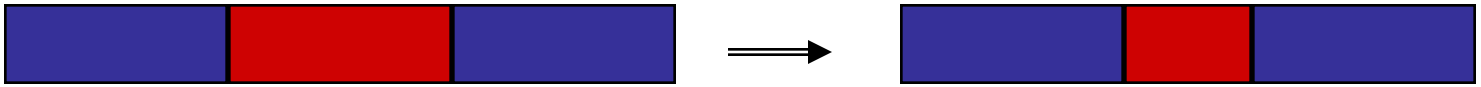


Make the common case fast: Amdahl's Law (§ 1.9)

Speedup due to some enhancement E:

$$Speedup_{overall} = \frac{ExTime_{withoutE}}{ExTime_{withE}} = \frac{Performance_{withE}}{Performance_{withoutE}}$$

Suppose that enhancement E accelerates a fraction of the task by a factor S, and the remainder of the task is unaffected



$$\frac{ExTime_{withE}}{ExTime_{withoutE}} = \left(1 - fraction_{enhanced}\right) + \frac{fraction_{enhanced}}{S}$$

Example: Floating point instructions can be improved to run 2X; but only 10% of actual instructions are FP. What is the overall speedup?

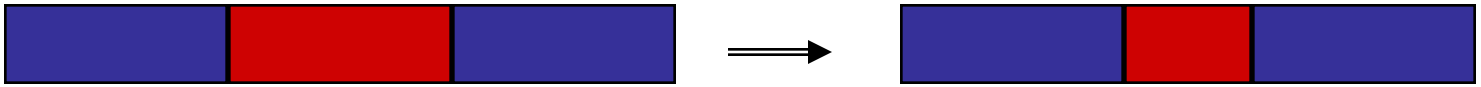
(1-10%) + 10%/2 = 95%. Speedup = 1/.95

Make the common case fast: Amdahl's Law (§ 1.9)

Speedup due to some enhancement E:

$$Speedup_{overall} = \frac{ExTime_{withoutE}}{ExTime_{withE}} = \frac{Performance_{withE}}{Performance_{withoutE}}$$

Suppose that enhancement E accelerates a fraction of the task by a factor S, and the remainder of the task is unaffected



$$\frac{ExTime_{withE}}{ExTime_{withoutE}} = \left(1 - fraction_{enhanced}\right) + \frac{fraction_{enhanced}}{S}$$

Example: How much faster must the multiplier be to get a 2x performance improvement overall if multiples account for 20 seconds of the 100 second run time?

Performance via Parallelism: Flynn's Taxonomy

SISD Single instruction stream Single data stream Single Core	(SIMD) Single instruction stream Multiple data stream GPUs, Vector Units
MISD Multiple instruction stream Single data stream TPUs	(MIMD) Multiple instruction stream Multiple data stream Multi-core

classic von Neumann

Does it make sense?
Yes, systolic array.